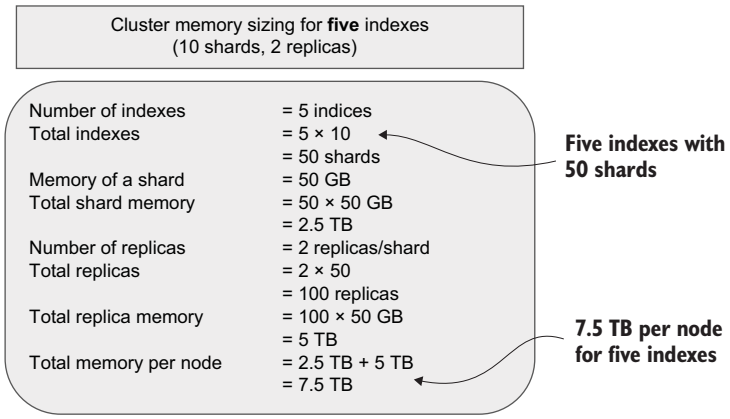


Figure 14.8 Exponential memory usage for five indexes on a node

As you can see, memory increases exponentially when we consider additional indexes. In the earlier case of one index with 10 shards and 20 replicas (figure 14.7), we commissioned a 5-node cluster to handle 2000 GB of space requirements. In this new case with multiple indexes (figure 14.8), we need a massive cluster that can handle about 10 TB of space. We can deal with this in two ways: vertical scaling or horizontal scaling.

VERTICAL SCALING

We can use the same cluster that serves the 2 TB memory requirements and prop up the additional memory to handle new memory requirements: for example, increasing each server to 2 TB (five nodes with 2 TB = 10 TB). This is a vertical scaling activity. Although this is not a problem on a technical level, we may need to take the server down to upgrade it.

HORIZONTAL SCALING

The alternate (and probably preferred) approach is to add nodes to the cluster. For example, we can add 20 more nodes to the server, for a total of 25 nodes. We then have a newly formed cluster with 25 nodes, where each node has 400 GB of memory: the total can handle our 10 TB memory requirements.

While there is no one-size-fits-all solution, having a forward-thinking strategy with a tried-and-tested approach works well for most organizations. Shard sizing is tedious, and we need to exercise extreme caution to size them appropriately.

Because a major administrative requirement is making backups of indexes or the entire cluster from time to time, we cover this in the next section. Additionally, we look at restoring backups when needed. Elasticsearch provides a sleek mechanism for backing up and restoring—the snapshot.

14.4 Snapshots

Running applications in production without backup and restore functionality is risky. Data in our clusters should be stored in durable storage somewhere off-cluster. Fortunately, Elasticsearch provides easy snapshot and restore functionality for backing up our data and restoring it when needed.